# Column counts in MULTIZ: aligning two alignments in $K + L$, not $K \times L$

Richard Burhans
Penn State University ORCID 0009-0006-5949-0958

Technical note, 5 November 2026

**Abstract**

MULTIZ builds genome-wide multiple alignments by repeatedly aligning one alignment to another. Its dynamic programming routine, `yama`, scored every cell by summing over all $K \times L$ pairs of rows, where $K$ and $L$ are the depths of the two blocks. On one deep merge from a 216-genome alignment, `yama` took 98 percent of the run time. Release 11.4 scores each cell from summaries made once per column instead: a score per symbol for one block and symbol counts for the other, plus counts of four gap classes. The work per cell no longer depends on the depths. Every score is an integer, so each total is the same as the pair-by-pair sum, and the output does not change. On synthetic blocks of $K = L = 128$ rows, `yama` itself ran about a thousand times faster and the whole merge 63 times faster, with identical output. The rest of the time is now spent scoring the merged blocks, which still visits every row pair. Neither half of the idea is new. Scoring a column from symbol counts is a profile, and counting gap states per column to avoid the $K \times L$ loop is Gotoh's method for aligning groups of sequences. This note records how the change was found, what it computes, where it comes from, and what it measured.

## 1 The problem

MULTIZ and the threaded blockset aligner build a multiple alignment progressively along a guide tree, and each step aligns two alignments that share a reference sequence [2]. The routine that does it, `yama` in `mz_yama.c`, runs a banded dynamic programming over the columns of the two blocks. Its objective is the sum of pairs: a column of block $A$ (with $K$ rows) aligned to a column of block $B$ (with $L$ rows) scores the sum of the substitution scores of all $K \times L$ row pairs. Gaps are charged by Altschul's quasi-natural gap costs, in the graph form of Chao, Hardison and Miller [1, 3]. Release 11.3 computed each cell's substitution sum and its three gap sums with loops over all $K \times L$ row pairs.

That cost grows with the product of the depths. Deep blocks are what a large panel produces. In a 216-genome alignment of *Cannabis sativa* assemblies to the cs10 reference, one merge aligned 2,303 blocks of about 35 rows to 4,359 blocks of about 43, so each cell visited about 1,505 row pairs, four times. Under `gprof`, `yama` took 98 percent of that merge's run time [9]. The change described here came from that profile, not from the literature.

## 2 The change

A row enters a cell's sums only through two things: its symbol in the current column, and whether it and the row's entry in the previous column are dashes. So each sum can be rebuilt from counts of

**Table 1: Synthetic block pairs, median of three runs per release.** Identical: the two outputs agree after removing # header lines.

| $K = L$ | $K \times L$ | v11.3 (s) | v11.4 (s) | speed-up | identical |
|---|---|---|---|---|---|
| 2 | 4 | 0.057 | 0.061 | 0.93 | yes |
| 4 | 16 | 0.113 | 0.059 | 1.92 | yes |
| 8 | 64 | 0.340 | 0.063 | 5.37 | yes |
| 16 | 256 | 1.282 | 0.076 | 16.84 | yes |
| 32 | 1,024 | 4.842 | 0.129 | 37.45 | yes |
| 64 | 4,096 | 19.412 | 0.322 | 60.35 | yes |
| 128 | 16,384 | 76.733 | 1.220 | 62.92 | yes |

those classes (Fig. 1).

**Substitution.** For each column of $A$, release 11.4 computes once a score for every symbol $s$,

$$w_A(s) = \sum_{i=1}^{K} \sigma(a_i, s),$$

where $\sigma$ is the substitution matrix. For each column of $B$ it counts the rows holding each symbol, $n_B(s)$. The cell's substitution sum is then $\sum_s n_B(s)\, w_A(s)$, a dot product over the five symbols A, C, G, T and N. Preparing $w_A$ costs $K$ steps per column of $A$ (times the symbol count), and $n_B$ costs $L$ per column of $B$. Neither is repeated per cell.

**Gaps.** Under quasi-natural gap costs, whether a pair of rows opens a gap at a new column is decided by the preceding column alone. Altschul defined them that way, adding one gap in the case where natural costs would need more history, so that "the preceding edge completely determines the number of gaps a new edge introduces" [1]. A row's contribution to the gap sums therefore depends only on two flags: is it a dash here, and was it a dash in the previous column. Each row falls into one of four classes, and the gap sums are products of class counts from the two blocks.

**Exactness.** All scores are integers. Each total is the pair-by-pair sum regrouped, so it is the same number, and the traceback makes the same choices. The output of 11.4 is identical to that of 11.3 apart from the # header lines (Section 3).

## 3 What it measured

**Synthetic blocks.** To measure the scaling on public, reproducible input, we built 11.3 and 11.4 from their release tags with identical flags (`-O2`) and merged synthetic block pairs with $K = L$ from 2 to 128. Each pair has 10 blocks of 2,000 reference columns, a shared reference row, species rows with 2 percent substitutions and 1 percent deletions, and insertion columns placed differently in the two files, so `yama` has columns to align. Times are medians of three runs (Table 1, Fig. 2). The 11.3 time rose about four times for each doubling of $K$, as $K \times L$ predicts, from 0.34 s at $K = 8$ to 76.7 s at $K = 128$. The 11.4 time stayed between 0.06 and 0.13 s up to $K = 32$ and reached 1.22 s at 128. The speed-up was 5.4-fold at $K = 8$, 37-fold at 32 and 63-fold at 128. Outputs were identical at all seven depths.

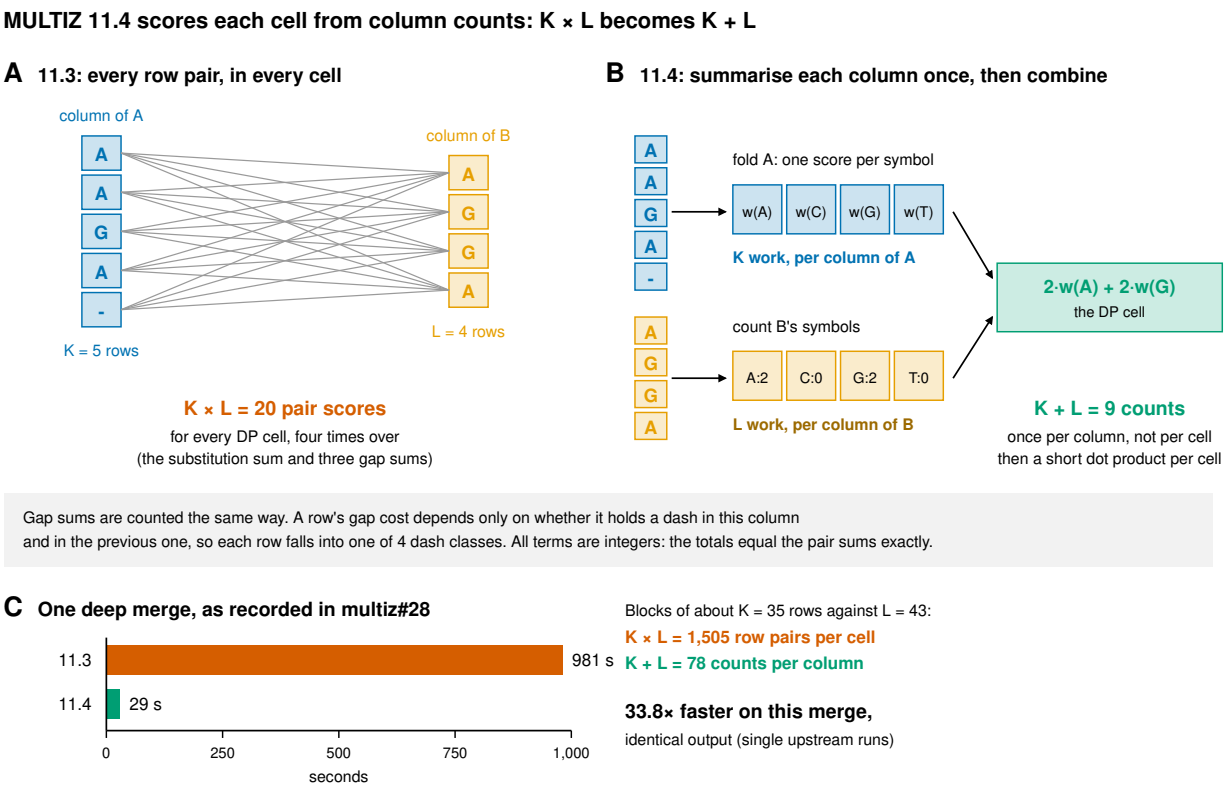



**Figure 1: Column counts turn $K \times L$ work per cell into $K + L$ work per column.** In panel A, release 11.3 scored a toy column pair of $K = 5$ and $L = 4$ rows by visiting all 20 row pairs, once for the substitution sum and once for each of three gap sums. In panel B, release 11.4 folds the column of $A$ into one score per symbol and counts the symbols of $B$'s column, each once per column. The cell is their dot product, here $2w(\mathrm{A}) + 2w(\mathrm{G})$. Gap sums are counted by four dash classes. Blue marks $A$, orange $B$, green the 11.4 cell and vermilion the 11.3 row-pair work. Panels A and B are a toy example, not data. Panel C is the deep merge recorded in `multiz#28`, one run per release.

**Product, not square.** With $K = L$ the sweep cannot tell $K \times L$ from $K^2$. A grid of unequal depths, $K$ and $L$ each 4, 16 or 64, separates them (Fig. 3). The 11.3 time depended on the product alone. At $K \times L = 256$, the $4 \times 64$, $16 \times 16$ and $64 \times 4$ pairs took 1.27, 1.25 and 1.46 s, and at 1,024 the $16 \times 64$ and $64 \times 16$ pairs took 4.83 and 4.80 s, against 4.84 s for $32 \times 32$. All nine outputs were identical.

**Where the time goes now.** The 11.4 time does not stay proportional to $K + L$ at depth. From $K = L = 64$ to 128 it rose 3.8-fold. A `gprof` profile of the $K = L = 128$ merge shows why (`measurements/gprof_k128.txt`). In 11.3, `yama` took 76.7 s of 77.8 and `mafScoreRange` 1.1 s. In 11.4, `yama` took 0.06 s, about a thousand times less, while `mafScoreRange` still took 1.1 s, now 94 percent of the run. `mafScoreRange` scores each merged block by summing over all pairs of its $K + L - 1$ rows, so it grows with the square of the total depth. That caps the speed-up near 60-fold in this sweep. The same counting would apply to it, which makes it the next target.

**Real merges.** The pull request that introduced the change records three more measurements [9]. The deep merge above took 981 s with the 11.3 release build and 29 s with 11.4, and its output and both files of unused blocks were identical apart from the header line. A whole 216-genome roast of one 4.4 Mb *Cannabis* piece took a median 14,085 s with 11.3 and 2,873 s with 11.4 (4.9-fold,

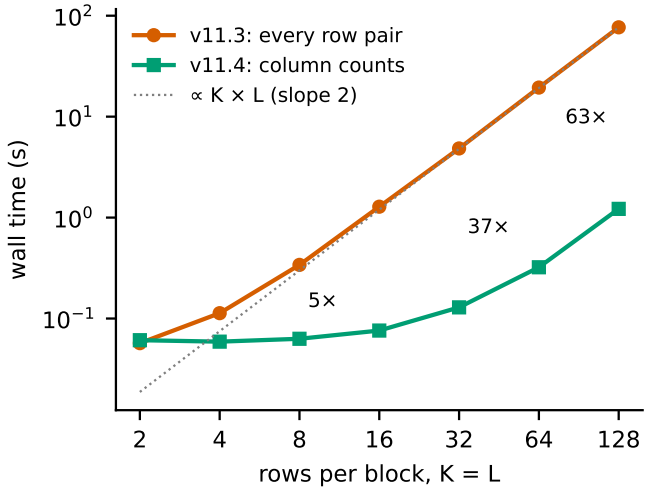



**Figure 2: The 11.3 time follows $K \times L$, and the 11.4 time grows far more slowly.** Wall time of one `multiz` merge of a synthetic block pair against block depth, log-log, median of three runs. Vermilion circles are v11.3, green squares v11.4, and the dotted line has slope 2 through the last v11.3 point. Labels give the speed-up. Below $K = 4$ both releases spend their time outside `yama`.

4.5- to 5.2-fold across single-run pairings), three interleaved runs per release on one host, all six outputs identical apart from # lines. UCSC's hg38 `multiz5way` recipe still reproduced the published alignment, 7 of 7 sequences.

**Why a whole run gains less.** A roast merges along the whole guide tree, and the merges low in the tree are shallow. At shallow depth `yama` is not the bottleneck: in the sweep, 11.3 and 11.4 differ by under 2-fold below $K = 8$. Block scoring, projection and file handling also take time that this change does not touch. So the gain on a whole run depends on how much of it is spent in deep merges, which is why one deep merge gained 33.8-fold and a whole piece 4.9-fold.

## 4 Prior work

Neither half of the change is new.

Scoring a column from symbol counts is a profile. Gribskov, McLachlan and Eisenberg defined the profile score of symbol $a$ at position $p$ as $M(p, a) = \sum_b W(p, b)\, Y(a, b)$, with $Y$ the substitution matrix and the simple weight $W(p, b)$ the count of residue $b$ in that column divided by the number of sequences [6]. The fold $w_A$ above is the same sum without the division.

Counting gap states per column to escape the $K \times L$ loop is Gotoh's. Gotoh gave exact sum-of-pairs algorithms for aligning two groups of sequences [4], and then removed their cost in proportion to the product of the group sizes. In his words, profile vectors could already reduce the work, but "there has been no way to evaluate the accurate gap-opening costs with computation steps less than $O(M \times N)$" [5]. He did it with gap-state variables computed once per column and combined during the recursion, and reported speed-ups of about 10-fold when $M \times N \approx 200$ and over 100-fold when $M \times N > 2{,}500$. His gap states track runs of any length, because his objective counts gaps exactly. `yama`'s quasi-natural costs look back one column, so the states reduce to four classes.

Exact gap counting is what makes the general problem hard. With linear gap costs counted exactly, aligning two alignments under the sum-of-pairs objective is NP-complete [8], and Kececioglu

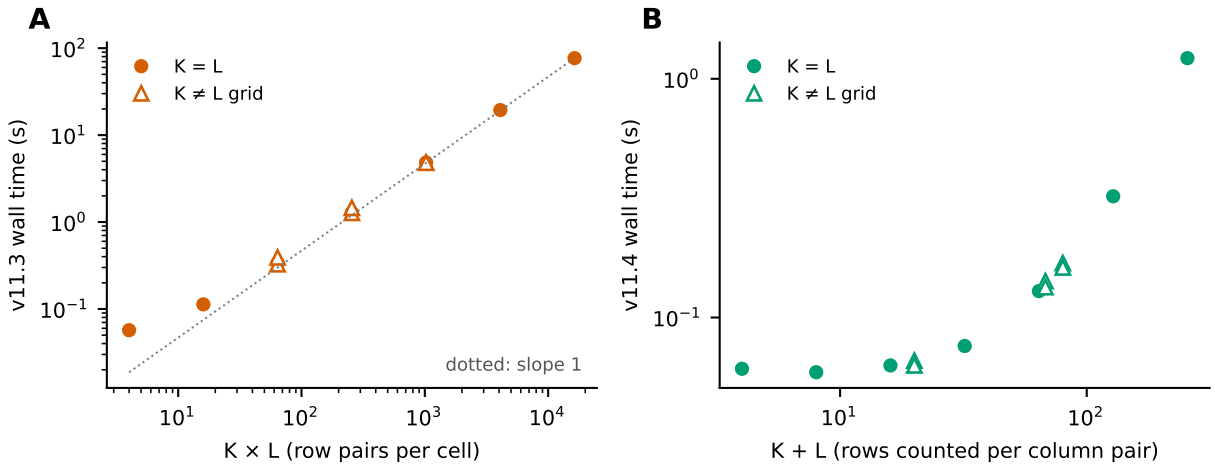



**Figure 3: The 11.3 time depends on $K \times L$, whatever the shape of the pair.** Panel A plots the v11.3 wall time against $K \times L$ and panel B the v11.4 time against $K + L$, both log-log. Filled circles are the $K = L$ sweep, open triangles the grid with $K \neq L$ (each of 4, 16 and 64). The dotted line in A has slope 1. Medians of three runs, all outputs identical. In B the time is flat at about 0.06 s for small depths, where little time is spent aligning, and grows faster than $K + L$ at large depths, where `mafScoreRange` dominates.

and Zhang had earlier relaxed it with optimistic and pessimistic gap counts [7]. Release 11.4 does not touch that problem. It computes `yama`'s existing objective, quasi-natural gaps included, exactly and faster.

A search on 5 November 2026 (`measurements/prior_art_search.md`) found no publication that applies this reduction to MULTIZ or the threaded blockset aligner.

## 5 Limits

- The sweep's blocks are synthetic: uniform random sequence, fixed mutation rates, one seed, equal depths $K = L$. They show how the cost scales with depth, not how a real alignment behaves.
- The sweep ran on a 32-vCPU host while unrelated single-threaded alignment jobs were running (load average about 4.8). Each `multiz` run is single-threaded, and the two releases ran back to back for each size.
- The profiles are `gprof` samples at 0.01 s, so the 0.06 s for `yama` in 11.4 is coarse. It supports "about a thousandfold", not a precise ratio.
- The deep-merge and whole-piece timings are taken from the pull request and were not rerun for this note. The deep merge is one run per release.
- Output identity was checked on the seven synthetic pairs, the deep merge, the whole piece and the hg38 5-way recipe. It is an empirical check of the exactness argument, not a proof.

## Methods and availability

`probe/build_multiz.sh` clones `multiz` at tags `v11.3` (commit `645ab55`) and `v11.4` (`6bfe6a4`) and builds both with `-O2 -Wall -Wextra` (gcc 15.2.0). `probe/make_blocks.py` writes the synthetic pairs

(seed 1), and `probe/run_sweep.py` times them and compares outputs, writing `measurements/sweep.csv` ($K = L$) and `measurements/grid.csv` (`--sizes 4x4,4x16,...,64x64`). `probe/profile_k128.sh` builds both releases with `-pg` and writes `measurements/gprof_k128.txt`. The host is recorded in `measurements/sweep_env.txt`. Figures are made by `paper/make_figure1.py`, `paper/make_figure2.py` and `paper/make_figure3.py`. MULTIZ is at https://github.com/multiz/multiz. Text, figures and code are CC0 1.0.

## Acknowledgements

Minmei Hou suggested describing the change as reducing $K \times L$ work to $K + L$.

## References

[1] S. F. Altschul. Gap costs for multiple sequence alignment. *Journal of Theoretical Biology* 138:297–309, 1989. doi:10.1016/S0022-5193(89)80196-1.

[2] M. Blanchette, W. J. Kent, C. Riemer, et al. Aligning multiple genomic sequences with the threaded blockset aligner. *Genome Research* 14:708–715, 2004. doi:10.1101/gr.1933104.

[3] K.-M. Chao, R. C. Hardison and W. Miller. Recent developments in linear-space alignment methods: a survey. *Journal of Computational Biology* 1:271–291, 1994. doi:10.1089/cmb.1994.1.271.

[4] O. Gotoh. Optimal alignment between groups of sequences and its application to multiple sequence alignment. *Computer Applications in the Biosciences* 9:361–370, 1993. doi:10.1093/bioinformatics/9.3.361.

[5] O. Gotoh. Further improvement in methods of group-to-group sequence alignment with generalized profile operations. *Computer Applications in the Biosciences* 10:379–387, 1994. doi:10.1093/bioinformatics/10.4.379.

[6] M. Gribskov, A. D. McLachlan and D. Eisenberg. Profile analysis: detection of distantly related proteins. *Proceedings of the National Academy of Sciences USA* 84:4355–4358, 1987. doi:10.1073/pnas.84.13.4355.

[7] J. Kececioglu and W. Zhang. Aligning alignments. In *Combinatorial Pattern Matching* (*CPM 1998*), Lecture Notes in Computer Science, 189–208, 1998. doi:10.1007/BFb0030790.

[8] J. Kececioglu and D. Starrett. Aligning alignments exactly. In *Proceedings of the Eighth Annual International Conference on Research in Computational Molecular Biology* (*RECOMB 2004*), 85–96, 2004. doi:10.1145/974614.974626.

[9] multiz/multiz pull request 28, "yama: build each cell's sums from column counts, not row pairs" (2026). https://github.com/multiz/multiz/pull/28.